查找算法记忆树

> 本质：查找算法，就是在大量数据中“尽快找到目标值”。
> 关键区别在于：是否有序、是否能直接定位、是否适合随机访问。

一、查找算法总览

- 顺序查找
- 核心：从头到尾逐个比较
- 特征：
- 最简单，最直观
- 不要求数据有序
- 适合小规模数据或链表
- 时间复杂度：平均/最坏 O(n)，最好 O(1)
- 空间复杂度：O(1)
- 适用场景：数据量小、无序数组、链表遍历
- 一句话：**一个一个看，直到找到**
- 二分查找
- 核心：有序数组中，每次缩小一半范围
- 特征：
- 只适用于有序序列
- 每次砍一半，查找效率高
- 需要随机访问下标
- 时间复杂度：最好 O(1)，平均 O(log n)，最坏 O(log n)
- 空间复杂度：O(1)（循环版）；O(log n)（递归版）
- 适用场景：有序数组、查找频繁、随机访问高效
- 一句话：**有序就砍一半**
- 插值查找
- 核心：不是固定中间点，而是根据目标值估算更接近的位置
- 特征：
- 适用于“均匀分布”的有序数组

- 在数据分布比较均匀时，比二分查找更快
- 若数据分布不均匀，优势会变小
- 时间复杂度：最好 O(1)，平均 O(log log n)，最坏 O(n)
- 空间复杂度：O(1)
- 适用场景：有序且分布均匀的数据，尤其是大规模数组
- 一句话：**按比例猜位置**
- 斐波那契查找
- 核心：用斐波那契数列划分查找区间，类似于二分查找的改进版
- 特征：
- 也要求有序数组
- 比二分查找更“贴近分段”
- 适合对数级搜索的高级变体
- 时间复杂度：平均/最坏 O(log n)
- 空间复杂度：O(1)
- 适用场景：理论研究、改进型二分查找场景
- 一句话：**用斐波那契分段找**
- 哈希查找
- 核心：通过哈希函数把 key 映射到存储位置，直接定位
- 特征：
- 平均查找接近 O(1)
- 依赖哈希函数设计和冲突处理
- 支持“按值查找”很快
- 时间复杂度：平均 O(1)，最坏 O(n)
- 空间复杂度：O(n)
- 适用场景：字典、缓存、Map、对象属性查找、海量数据快速定位
- 一句话：**算地址，直接找**
- 二叉排序树查找（BST）
- 核心：利用“左小右大”的特点，递归/循环查找
- 特征：
- 适合动态插入删除
- 平均情况下查找效率很好
- 最坏退化成链表

- 时间复杂度：平均 O(log n)，最坏 O(n)
- 空间复杂度：O(h)，其中 h 是树高
- 适用场景：动态数据、需要频繁插入/删除的查询场景
- 一句话：**左小右大，往树里走**
- B 树 / B+ 树查找
- 核心：多路平衡搜索树，适合磁盘/外存场景
- 特征：
- 一个节点存多个关键字
- 适合大规模数据的块式存储
- 常见于数据库和文件系统
- 时间复杂度：平均/最坏 O(log_B n)
- 空间复杂度：O(n)
- 适用场景：数据库索引、文件系统、外存数据管理
- 一句话：**一层放很多关键字，适合大数据**

二、查找算法速查表

算法	是否有序	平均时间	最坏时间	空间复杂度	适用场景
顺序查找	否	O(n)	O(n)	O(1)	小规模、无序数据
二分查找	是	O(log n)	O(log n)	O(1) / O(log n)	有序数组
插值查找	是	O(log log n)	O(n)	O(1)	均匀分布数组
斐波那契查找	是	O(log n)	O(log n)	O(1)	理论/改进型二分
哈希查找	否（依赖键）	O(1)	O(n)	O(n)	字典、缓存、映射
BST 查找	依赖树结构	O(log n)	O(n)	O(h)	动态插入删除
B/B+ 树查找	是	O(log_B n)	O(log_B n)	O(n)	数据库、外存

三、查找算法记忆口诀

- 顺序查找：**一个一个找，直到找到**
- 二分查找：**有序数组，砍一半**

- 插值查找：**按比例估位置**
- 斐波那契查找：**分段像波浪**
- 哈希查找：**算地址，直接定位**
- BST 查找：**左小右大，树里走**
- B/B+ 树查找：**一页装很多，适合大数据**

四、重点判断口诀

1. 数据是否有序?

- 有序 → 可以考虑二分、插值、斐波那契
- 无序 → 一般只能顺序查找或哈希查找

2. 想不想“直接定位”?

- 想 → 哈希查找
- 不想 → 顺序/二分等

3. 需要动态插入删除?

- 需要 → BST、B/B+ 树
- 不需要 → 二分查找更简单

4. 数据大到要放磁盘/文件中?

- 需要 → B/B+ 树

五、考试常见易混点

1. 二分查找和顺序查找的区别

- 顺序查找：不要求有序，最简单
- 二分查找：要求有序，效率更高

2. 哈希查找和二分查找的区别

- 二分查找依赖“有序”；哈希查找依赖“映射函数”
- 哈希查找平均 O(1)，但最坏可能退化

- 二分查找稳定，最坏 O(log n)

3. BST 的最坏情况

- 如果一直插入有序数据，BST 会退化成链表
- 此时查找时间退化为 O(n)

4. B 树为什么适合数据库?

- 数据量大时，不能一层层从磁盘读很多小节点
- B 树和 B+ 树让一个节点存多个关键字，减少 I/O 次数

六、查找算法的核心理解

- 顺序查找：适合“谁都没排序”
- 二分查找：适合“已经排好序”
- 哈希查找：适合“按 key 直接定位”
- BST：适合“动态增加、删除、查询”
- B/B+ 树：适合“大数据 + 外存”

> 记忆一句：
> 顺序看一遍，二分砍一半，哈希直接算位置，树里左小右大，B 树一层装很多。

七、典型题型判断

- 题目说“有序数组” → 优先考虑二分查找
- 题目说“按编号查找、映射查找” → 考哈希查找
- 题目说“动态插入删除” → 考 BST 或 B/B+ 树
- 题目说“数据量大、外存” → 考 B/B+ 树
- 题目没有要求有序 → 优先顺序查找

八、专门补充：哈希表

哈希表是查找算法里非常重要的一类，详细内容请看：
哈希表_哈希查找.md

它重点讲的是：

- 哈希函数
- 哈希冲突
- 链地址法
- 开放定址法
- 平均查找复杂度 O(1)

九、最终速记版

- 查找 = 找谁?
- 顺序找：最简单
- 二分找：有序时最优
- 哈希找：直接定位
- BST 找：动态数据最合适
- B/B+ 树：数据库最常见

> 一句话记忆：
> “无序就遍历，有序就二分，直接定位用哈希，动态数据用树，大数据走 B 树。”